
Inteligência Artificial e Educação Matemática: contexto e desafios

1.1 Inteligência Artificial: definição operacional e classificações relevantes

O termo *inteligência artificial* foi empregado formalmente pela primeira vez em 1956, durante a Conferência de Dartmouth, convocada por [?] para reunir pesquisadores interessados em simular o raciocínio humano por meio de máquinas. Antes disso, Alan Turing já havia lançado, em 1950, uma questão fundadora — “*Can machines think?*” — e proposto um critério comportamental para avaliá-la: se uma máquina produz respostas indistinguíveis das humanas em uma interação textual mediada exclusivamente por escrita, considera-se que ela demonstra comportamento inteligente [?]. Não por acaso, a primeira aplicação formal dessa ideia, o *Logic Theorist* [?], foi construída para demonstrar teoremas da lógica proposicional — evidenciando, desde a origem, o vínculo estreito entre inteligência artificial e raciocínio matemático.

Para os fins desta dissertação, entretanto, adota-se uma definição funcional em lugar de uma genealogia histórica. Segundo russell2021, um sistema é dito inteligente quando percebe seu ambiente, processa informações e age de modo a maximizar a realização de objetivos previamente definidos. Essa perspectiva ancora a discussão no comportamento observável dos sistemas — e não em questões filosóficas sobre consciência ou cognição —, o que é suficiente e adequado ao contexto educacional aqui investigado.

Do ponto de vista das classificações, distinguem-se duas categorias relevantes para este trabalho. A primeira é a chamada **IA estreita** (*narrow AI*), que designa sistemas

projetados para executar tarefas específicas com alto desempenho — reconhecimento de imagens, tradução automática, sistemas de recomendação — sem capacidade de generalização além do domínio para o qual foram treinados [?]. A segunda é a **IA generativa**, categoria mais recente, que compreende modelos capazes de produzir conteúdo original — texto, imagem, código — a partir de padrões aprendidos em grandes volumes de dados [?]. É nessa segunda categoria que se inserem os sistemas centrais desta dissertação.

Para os propósitos deste trabalho, entende-se por “sistema de IA” qualquer ferramenta computacional baseada em modelos de linguagem de grande escala (*Large Language Models* — LLMs) capaz de gerar, avaliar ou organizar conteúdo textual de natureza matemática. Essa delimitação exclui, deliberadamente, sistemas voltados à visão computacional, robótica ou jogos, cujas arquiteturas e desafios diferem substancialmente do escopo aqui investigado.

1.2 IA generativa e modelos de linguagem de grande escala

A trajetória que levou aos atuais modelos de linguagem de grande escala tem início nas primeiras tentativas de simular a comunicação humana por meio de máquinas. Na década de 1960, weizenbaum1966 desenvolveu o ELIZA, programa estruturado para estabelecer diálogos por meio da identificação de padrões linguísticos e da reorganização das frases apresentadas pelo usuário. Ainda que não houvesse compreensão real do conteúdo, o ELIZA demonstrou que máquinas poderiam sustentar interações textuais convincentes e abriu caminho para o desenvolvimento de tecnologias conversacionais progressivamente mais sofisticadas.

Nas décadas seguintes, o avanço do poder computacional, a ampliação das bases de dados disponíveis e o desenvolvimento de métodos conexionistas — especialmente as redes neurais artificiais — reposicionaram a inteligência artificial como campo estratégico de pesquisa [?]. Nesse contexto, bengio2003 propuseram os primeiros modelos neurais de linguagem capazes de representar palavras como vetores em espaços semânticos contínuos, estabelecendo as bases conceituais para o que viria a seguir.

O avanço decisivo ocorreu em 2017, com a introdução da arquitetura *transformer* por vaswani2017. Seu mecanismo central — a *atenção (self-attention)* — permite que o modelo pondere, a cada etapa da geração, quais partes de um texto de entrada são mais relevantes para produzir o próximo trecho de saída. Essa capacidade de capturar dependências de longo alcance em sequências textuais representou uma ruptura em relação

às arquiteturas recorrentes anteriores, permitindo o treinamento em volumes de dados e escalas computacionais sem precedentes.

Um modelo de linguagem, em sentido técnico, é um sistema que aprende a distribuição de probabilidade sobre sequências de tokens — unidades mínimas de texto, que podem corresponder a palavras, subpalavras ou caracteres. Durante o treinamento, o modelo é exposto a grandes corpora textuais e ajusta seus parâmetros internos para maximizar a probabilidade de prever o próximo token dado o contexto anterior [?]. Na fase de inferência, a geração ocorre de forma autorregressiva: a cada passo, o modelo produz uma distribuição de probabilidade sobre o vocabulário e seleciona o próximo token com base nessa distribuição, acumulando o resultado até completar a resposta.

Os **modelos de linguagem de grande escala** (*Large Language Models* — LLMs) são a realização contemporânea desse princípio em escala massiva. Modelos como o GPT-3 [1], treinado com centenas de bilhões de parâmetros, demonstraram que o aumento de escala — em dados, parâmetros e capacidade computacional — produz capacidades emergentes não observadas em modelos menores: raciocínio analógico, geração de código, tradução entre idiomas e produção de texto de alta qualidade sem ajuste fino específico para cada tarefa. Essa característica tornou os LLMs a tecnologia dominante para geração de conteúdo textual na presente década.

Do ponto de vista desta dissertação, importa destacar uma propriedade central desses modelos: sua natureza **probabilística e não-determinística**. A geração de texto por LLMs não é o resultado de regras simbólicas explícitas, mas de previsões estatísticas construídas a partir de padrões latentes nos dados de treinamento [3]. Essa característica confere aos modelos grande versatilidade — inclusive para gerar questões matemáticas originais —, mas também os torna suscetíveis a erros sistemáticos em tarefas que exigem rigor lógico e consistência simbólica, aspecto que será tratado em detalhe na Seção ??.

1.3 IA na Educação: panorama de aplicações

A incorporação da inteligência artificial ao campo educacional configura-se como um processo mais complexo e gradual do que o observado em outros setores. Diferentemente do ambiente empresarial, cuja adoção é fortemente impulsionada por demandas de produtividade e competitividade, a educação envolve dimensões pedagógicas, éticas e sociais que exigem reflexão crítica e regulamentação cuidadosa [?]. Questões relacionadas à formação docente, à infraestrutura tecnológica, à proteção de dados de estudantes e às implicações didáticas do uso dessas ferramentas tornam o avanço necessariamente mais cauteloso. Assim, embora a IA apresente potencial significativo para apoiar

práticas pedagógicas e personalizar a aprendizagem, sua incorporação ao contexto escolar ainda ocorre de maneira progressiva e permeada por debates acerca de seus limites e possibilidades [2].

Dentre as categorias de aplicação mais estudadas na literatura, os **sistemas tutoriais inteligentes** (*Intelligent Tutoring Systems* — ITS) figuram entre as mais antigas e consolidadas. Esses sistemas buscam simular a interação individualizada entre professor e aluno, identificando lacunas no conhecimento do estudante e adaptando a sequência de atividades em tempo real [?]. A motivação teórica remonta ao estudo seminal de bloom1984, que demonstrou que o atendimento individualizado produz ganhos de aprendizagem significativamente superiores aos do ensino coletivo convencional — o chamado *problema das duas sigmas*. Pesquisas posteriores indicam que ITS bem projetados conseguem aproximar-se desse referencial, com efeitos positivos documentados especialmente em matemática e ciências [?].

Uma segunda categoria relevante compreende os **sistemas de recomendação e ambientes adaptativos**. Nesses sistemas, algoritmos de aprendizado de máquina analisam o histórico de interações do estudante — desempenho em atividades, tempo de resposta, padrões de erro — e recomendam conteúdos, exercícios ou percursos de aprendizagem personalizados [?]. A lógica subjacente é semelhante à dos sistemas de recomendação comerciais, mas orientada a objetivos pedagógicos: maximizar a progressão do estudante respeitando seu ritmo e suas dificuldades específicas. Plataformas como Khan Academy e Duolingo incorporam variações dessa abordagem em larga escala.

A **geração automática de conteúdo educacional** constitui a terceira categoria e a mais diretamente relacionada ao objeto desta dissertação. Com o advento dos LLMs, tornou-se tecnicamente viável produzir questões, enunciados, resumos, feedback personalizado e até planos de aula de forma automatizada, a partir de especificações fornecidas pelo professor [?]. Essa possibilidade tem despertado interesse crescente na pesquisa em Educação Matemática, pois pode reduzir o tempo dedicado à elaboração de materiais e ampliar a diversidade de situações-problema disponíveis para o trabalho em sala de aula. Entretanto, como será discutido na Seção ??, a geração automática em contextos matemáticos apresenta fragilidades específicas que demandam estratégias de verificação e validação docente.

Por fim, os **assistentes conversacionais** representam a categoria de maior penetração no cotidiano de professores e estudantes na atualidade. Ferramentas baseadas em LLMs — como o ChatGPT, o Gemini e o próprio Claude — são amplamente utilizadas para tirar dúvidas, solicitar explicações alternativas, gerar exemplos e apoiar a resolução de problemas [?]. Sua acessibilidade, disponibilidade contínua e capacidade de adaptar

o nível de linguagem ao interlocutor conferem-lhes potencial pedagógico relevante. Ao mesmo tempo, sua natureza probabilística — que pode produzir respostas plausíveis, porém incorretas, sem qualquer sinalização de incerteza — impõe ao professor o papel insubstituível de mediador crítico, tema desenvolvido na Seção ??.

1.4 IA no ensino de Matemática: potencialidades e fragilidades

A chegada de modelos de linguagem de grande escala ao cotidiano escolar suscita tanto entusiasmo quanto cautela entre educadores e pesquisadores. No campo da Matemática, essa ambivalência é especialmente marcada: ao mesmo tempo que os *Large Language Models* (LLMs) demonstram capacidade notável de comunicar ideias matemáticas em linguagem natural e de produzir resoluções passo a passo, eles exibem falhas sistemáticas em tarefas que deveriam ser triviais para qualquer computador convencional [?, ?].

Potencialidades reconhecidas

Do ponto de vista pedagógico, os LLMs oferecem três contribuições que merecem destaque. A primeira é a **multiplicidade de abordagens explicativas**: diante de uma mesma questão, o modelo pode reformular o enunciado, apresentar exemplos análogos, oferecer representações visuais em *ASCII* ou sugerir estratégias heurísticas alternativas, adaptando a linguagem ao perfil do estudante. Essa flexibilidade aproxima-se do ideal de ensino diferenciado preconizado pela literatura em educação matemática [?].

A segunda potencialidade é a **geração rápida de material didático**. Um professor pode, em poucos minutos, obter rascunhos de enunciados, variações de um mesmo problema ou conjuntos de exercícios graduados por dificuldade. Plataformas comerciais como Khan Academy e Coursera já exploram essa capacidade para reduzir a carga de criação de conteúdo e ampliar a personalização do ensino [?].

A terceira contribuição é o **suporte à articulação verbal do raciocínio**. Ao exigir que o modelo apresente cada etapa de resolução, a técnica de *chain-of-thought prompting* — introduzida por wei2022chain — torna o processo de solução legível e discutível, o que pode ser aproveitado pelo professor como material para análise e debate em sala. Ao explicitar passos intermediários antes de enunciar a resposta final, essa abordagem melhora significativamente o desempenho dos modelos em tarefas de raciocínio aritmético e algébrico, aproximando a saída do modelo de uma demonstração comentada [?, ?].

Fragilidades documentadas

As limitações dos LLMs em Matemática são igualmente documentadas e, para os fins desta dissertação, mais relevantes do que as potencialidades. frieder2023mathematical conduziram uma avaliação sistemática das capacidades matemáticas do ChatGPT/GPT-4, concluindo que o modelo apresenta desempenho aceitável em questões de nível de graduação, mas falha de forma expressiva em problemas que exigem demonstração formal, manipulação simbólica complexa ou raciocínio em múltiplos passos encadeados. Essa assimetria entre fluência discursiva e rigor computacional é uma das características mais relevantes para o uso educacional da tecnologia.

Quatro categorias de fragilidade são recorrentes na literatura:

1. **Erros aritméticos e algébricos básicos.** Estudos recentes mostram que LLMs cometem erros surpreendentes em tarefas de compreensão numérica fundamental, incluindo operações que qualquer calculadora executa sem falha [?, ?]. Simplificações algébricas incorretas e equívocos em contagens figuram entre os padrões de erro mais frequentes.
2. **Raciocínio em múltiplos passos.** À medida que a cadeia de resolução se alonga, a probabilidade de propagação de erros cresce de forma cumulativa. Um equívoco em uma etapa intermediária invalida todas as etapas subsequentes, ainda que cada passo isolado pareça plausível [?]. Esse fenômeno é descrito na literatura como *cascading error* e representa um risco concreto em qualquer aplicação que utilize LLMs para resolver ou gerar problemas matemáticos encadeados.
3. **Álgebra simbólica formal.** A manipulação de expressões simbólicas — fatoração, simplificação de frações algébricas, resolução de sistemas por substituição — exige precisão determinística que os LLMs, por sua natureza probabilística, não garantem [?]. Sistemas de computação algébrica (CAS), como o SymPy, são projetados exatamente para esse tipo de tarefa e apresentam desempenho qualitativamente superior.
4. **Sensibilidade ao fraseado do *prompt*.** Pequenas alterações na forma como uma questão é enunciada podem produzir respostas radicalmente diferentes, inclusive contraditórias entre si. Isso indica que o modelo frequentemente responde ao padrão superficial do texto, e não ao conteúdo matemático subjacente [?].

O fenômeno do erro silencioso

A principal implicação pedagógica das fragilidades acima não é a possibilidade de erro em si, mas o modo como esse erro se apresenta: o LLM tende a produzir respostas **plausíveis e incorretas**, redigidas com o mesmo tom de confiança das respostas corretas. Diferentemente de uma calculadora, que simplesmente rejeita uma entrada inválida, ou de um sistema CAS, que emite uma mensagem de erro, o LLM elabora uma explicação coerente para um resultado equivocado. Esse fenômeno — denominado *alucinação* na literatura de processamento de linguagem natural e aqui chamado de **erro silencioso** por sua invisibilidade ao leitor não especializado — representa um risco específico no contexto educacional, onde o professor ou o estudante pode aceitar um gabarito incorreto sem perceber o equívoco [?, ?].

Implicações para esta dissertação

O diagnóstico apresentado nesta seção fundamenta diretamente a hipótese central deste trabalho: o uso isolado de um LLM para geração de questões matemáticas é insuficiente para garantir correção e qualidade didática. A combinação do poder expressivo dos LLMs com a verificação determinística de um sistema de computação algébrica e com a avaliação pedagógica estruturada de um agente especializado configura, portanto, não um refinamento opcional, mas uma necessidade arquitetural. Os mecanismos técnicos que materializam essa combinação são detalhados no Capítulo ??.

Fundamentação técnica: LLMs, agentes e verificação simbólica

Descrição: capítulo técnico-central da fundamentação. Apresenta com profundidade os três pilares conceituais do sistema: LLMs, sistemas multiagentes e verificação simbólica via CAS. Justifica tecnicamente por que a combinação desses elementos é necessária.

2.1 Modelos de linguagem de grande escala

Os modelos de linguagem de grande escala (*Large Language Models* — LLMs) são sistemas computacionais treinados para processar e gerar texto em linguagem natural a partir de padrões aprendidos em volumes massivos de dados textuais. Compreender seu funcionamento em nível conceitual é condição necessária para avaliar tanto suas potencialidades quanto suas limitações no contexto educacional e, em particular, para justificar por que a arquitetura proposta nesta dissertação requer camadas adicionais de verificação simbólica e avaliação pedagógica.

2.1.1 Arquitetura *transformer* em nível conceitual

A quase totalidade dos LLMs contemporâneos é construída sobre a arquitetura *transformer*, introduzida por vaswani2017attention no artigo *Attention Is All You Need*. Antes dessa proposta, os modelos de linguagem predominantes utilizavam redes neurais recorrentes (RNNs), que processavam sequências de forma estritamente serial — um elemento de cada vez — e, por isso, apresentavam dificuldades em capturar dependências entre palavras muito distantes no texto. A contribuição central dos *transformers* foi

substituir essa lógica sequencial por um mecanismo de **autoatenção** (*self-attention*), que permite ao modelo avaliar, simultaneamente, a relevância de cada elemento de uma sequência em relação a todos os demais [?]. Essa capacidade de processamento paralelo tornou o treinamento de modelos muito maiores viável e deu origem à atual geração de LLMs [?].

Para os fins desta dissertação, a arquitetura pode ser compreendida a partir de três conceitos encadeados. O primeiro é o de **token**: a unidade mínima de processamento do modelo, que não corresponde necessariamente a uma palavra inteira, mas a fragmentos de texto determinados estatisticamente durante o treinamento [?]. A expressão "*função quadrática*", por exemplo, pode ser segmentada em dois, três ou mais tokens, dependendo do vocabulário do modelo. Crucialmente, o modelo opera inteiramente sobre sequências de identificadores numéricos associados a esses tokens — jamais sobre o texto em si. Isso implica que toda a sofisticação linguística de um LLM é, em sua raiz, uma transformação de sequências de números.

O segundo conceito é o de **janela de contexto**: o número máximo de tokens que o modelo pode considerar em uma única operação de inferência. Tudo que precisa ser levado em conta na geração de uma resposta — o enunciado da tarefa, o histórico da conversa, os exemplos fornecidos e a própria resposta em construção — deve caber dentro dessa janela [?]. Modelos com janelas mais amplas conseguem manter coerência em textos mais extensos, o que é relevante para a geração de questões com contextos elaborados e resoluções detalhadas.

O terceiro conceito é o próprio mecanismo de **atenção**: o componente pelo qual o modelo pondera a influência de cada token sobre os demais ao construir sua representação interna do texto [?]. É esse mecanismo que permite ao modelo reconhecer, por exemplo, que o pronome "*ela*" em uma frase retoma um sujeito mencionado vários tokens antes, ou que o termo "*raiz*" em um enunciado matemático pertence ao campo semântico algébrico, e não ao botânico. A atenção opera em múltiplas camadas e em múltiplas *cabeças* simultâneas, o que confere ao modelo capacidade de capturar relações de diferentes naturezas em paralelo.

2.1.2 Inferência por previsão probabilística

Uma vez treinado, o LLM gera texto por meio de um processo denominado **inferência autorregressiva**: a cada passo, calcula-se uma distribuição de probabilidade sobre todos os tokens do vocabulário e seleciona-se o próximo token com base nessa distribuição. O token escolhido é acrescentado à sequência, que passa a servir de contexto para a previsão seguinte, e o processo se repete até que um critério de parada seja atingido [?].

Essa natureza probabilística tem uma consequência fundamental que distingue os LLMs de qualquer ferramenta de cálculo convencional: **o modelo não computa, ele prevê**. Quando solicitado a resolver a equação $2x^2 - 5x + 3 = 0$, o LLM não executa um algoritmo de fatoração; ele gera a sequência de tokens que, dado seu treinamento, tem maior probabilidade de aparecer após aquele enunciado. Na maior parte dos casos comuns, esse mecanismo produz respostas corretas, pois o treinamento expôs o modelo a inúmeros exemplos semelhantes. Porém, ele não oferece nenhuma garantia de correção formal, o que o diferencia estruturalmente de um sistema de computação algébrica.

Dois parâmetros controlam o grau de variabilidade da geração. A **temperatura** escala a distribuição de probabilidade antes da amostragem: valores próximos de zero tornam o modelo mais determinístico, favorecendo o token mais provável; valores elevados achatam a distribuição, aumentando a probabilidade de tokens menos prováveis serem selecionados e, por isso, produzindo saídas mais variadas — e mais suscetíveis a erros em tarefas que exigem precisão [?]. O parâmetro **top-p** (*nucleus sampling*), proposto por holtzman2020curious, complementa esse controle ao restringir o conjunto de candidatos àqueles cuja probabilidade acumulada não ultrapassa um limiar definido, eliminando da amostragem os tokens de baixíssima probabilidade. O efeito prático é preservar diversidade sem abrir espaço para escolhas completamente improváveis [?].

Para aplicações matemáticas — onde a correção é binária — a variabilidade introduzida por esses parâmetros representa um risco concreto. Uma temperatura ligeiramente elevada pode fazer com que o modelo produza, em diferentes execuções do mesmo *prompt*, respostas distintas e contraditórias para um mesmo problema. Isso tem implicações diretas sobre a **reprodutibilidade** do sistema desenvolvido nesta dissertação, questão abordada no Capítulo ??.

2.1.3 Limitações conhecidas em raciocínio matemático

As limitações dos LLMs em tarefas matemáticas decorrem diretamente de sua arquitetura probabilística e foram documentadas de forma sistemática na literatura. Três categorias são especialmente relevantes para o presente trabalho.

A primeira é a **aritmética e a manipulação simbólica**. Como o modelo opera sobre tokens e não sobre estruturas algébricas, operações que exigem precisão formal — fatoração de polinômios, simplificação de frações algébricas, resolução de sistemas — não são computadas, mas aproximadas por padrão estatístico [?]. Quando o padrão é frequente no corpus de treinamento, o resultado tende a ser correto; quando envolve combinações menos recorrentes de valores ou estruturas, a probabilidade de erro cresce de forma não previsível.

A segunda limitação é o **raciocínio encadeado em múltiplos passos**. Em problemas que exigem uma sequência longa de deduções, cada etapa introduz uma nova fonte de variabilidade probabilística. Um equívoco em um passo intermediário não apenas invalida aquele passo isolado, mas contamina todos os subsequentes — o chamado *cascading error* discutido na Seção 1.4. A técnica de *chain-of-thought prompting* [?] atenua esse problema ao forçar a externalização dos passos intermediários, mas não o elimina: a cadeia de raciocínio pode ser fluente, coerente na forma e matematicamente incorreta no conteúdo.

A terceira limitação é a **sensibilidade ao fraseado do *prompt***. Pequenas variações na forma como um problema é enunciado podem produzir respostas estruturalmente diferentes, evidenciando que o modelo responde ao padrão superficial do texto em vez do conteúdo matemático subjacente [?]. Em um sistema de geração de questões, essa sensibilidade impõe que os *prompts* dos agentes sejam rigorosamente formulados e testados — uma decisão de projeto detalhada no Capítulo ??.

Em conjunto, essas três limitações fundamentam a hipótese central desta dissertação: nenhum LLM, utilizado de forma isolada, oferece as garantias necessárias para a geração confiável de questões matemáticas destinadas ao contexto escolar. A combinação com um verificador simbólico determinístico, apresentada na Seção ??, é a resposta arquitetural a esse diagnóstico.

2.2 Estratégias de mitigação documentadas na literatura

As limitações dos LLMs em raciocínio matemático, descritas na seção anterior, motivaram uma linha ativa de pesquisa voltada ao desenvolvimento de técnicas que aumentem a confiabilidade desses modelos sem exigir retreinamento. As quatro estratégias apresentadas a seguir representam o estado da arte nessa área e constituem o referencial técnico que orienta as decisões de projeto do sistema desenvolvido nesta dissertação.

2.2.1 *Chain-of-thought prompting*

A técnica de *chain-of-thought prompting* (CoT), proposta por wei2022chain, consiste em incluir, nos exemplos fornecidos ao modelo, não apenas o par pergunta-resposta, mas também a cadeia completa de passos intermediários que conduz à solução. A hipótese subjacente é que, ao ver exemplos de raciocínio explicitado, o modelo passa a produzir suas próprias cadeias de passos antes de enunciar a resposta final, o que reduz a probabilidade

de saltos inferências incorretos.

Os ganhos reportados por wei2022chain são expressivos: em benchmarks de raciocínio aritmético e lógico, o CoT superou abordagens de *few-shot* convencional com margem significativa, especialmente em modelos de maior escala. wang2023selfconsistency ampliaram esse resultado ao demonstrar melhorias adicionais de até 17,9 pontos percentuais no benchmark GSM8K quando o CoT é combinado com amostragem múltipla. Uma variante mais simples, o *zero-shot CoT*, foi proposta por kojima2022zero: basta acrescentar a instrução "*vamos pensar passo a passo*" ao *prompt* para que o modelo passe a externalizar seu raciocínio, sem necessidade de exemplos anotados.

Apesar desses avanços, o CoT apresenta limitações relevantes para o presente trabalho. A técnica melhora a coerência narrativa da resolução, mas não garante correção matemática: o modelo pode produzir uma cadeia de passos fluente, bem formatada e matematicamente incorreta. O erro pode ocorrer em qualquer etapa — um sinal trocado, uma fatoração equivocada, um caso desconsiderado — e, por estar embebido em um texto aparentemente coerente, tende a passar despercebido sem verificação independente [?]. É precisamente essa limitação residual que justifica a adição de uma camada de verificação simbólica no sistema desta dissertação.

2.2.2 *Program-aided language models (PAL)*

Uma abordagem mais radical ao problema da imprecisão matemática é a proposta por gao2023pal, denominada *Program-Aided Language Models (PAL)*. Em vez de solicitar ao modelo que resolva o problema diretamente, o PAL instrui o LLM a traduzir o enunciado em linguagem natural para um programa em Python, cujos passos de execução correspondem às etapas de raciocínio. A solução efetiva do problema é então **delegada a um interpretador externo** — determinístico e exato por natureza — que executa o código gerado e retorna o resultado.

Essa divisão de responsabilidades é conceitualmente simples e poderosa: o LLM faz o que faz bem (compreender linguagem natural e estruturar um plano de solução), e o interpretador faz o que o LLM não consegue garantir (computar com precisão). Nos experimentos de gao2023pal, o PAL superou modelos muito maiores que utilizavam apenas CoT, alcançando estado da arte no GSM8K com uma margem de 15 pontos percentuais sobre o PaLM-540B.

A conexão entre o PAL e a arquitetura desta dissertação é direta e constitui um dos pilares conceituais do trabalho. O Agente Verificador Simbólico, descrito no Capítulo ??, opera segundo o mesmo princípio fundamental: em vez de confiar na resolução textual produzida pelo Agente Gerador, ele delega o cômputo ao SymPy — um sistema de

computação algébrica simbólica — e compara o resultado obtido com o gabarito proposto. A diferença em relação ao PAL reside no escopo: onde o PAL utiliza um interpretador Python de propósito geral, o verificador desta dissertação emprega uma biblioteca especializada em álgebra simbólica, mais adequada às questões do Ensino Médio e capaz de lidar com equivalências formais que a aritmética numérica não captura.

2.2.3 *Tool use e function calling*

Uma generalização da ideia do PAL é o paradigma de *tool use*, que equipa o LLM com acesso a um conjunto de ferramentas externas — calculadoras, mecanismos de busca, APIs, interpretadores de código, sistemas de banco de dados — que podem ser invocadas durante a geração da resposta [?]. O modelo aprende não apenas a produzir texto, mas a decidir *quando* e *como* chamar cada ferramenta e a incorporar os resultados obtidos ao fluxo de geração.

schick2023toolformer demonstraram que modelos treinados com essa capacidade alcançam desempenho competitivo com modelos muito maiores em tarefas que envolvem aritmética, busca factual e consulta a calendários, sem sacrificar a fluência linguística. A implementação moderna dessa ideia em APIs comerciais de LLMs é o *function calling*: o modelo recebe a descrição de um conjunto de funções disponíveis e pode emitir chamadas estruturadas a essas funções no meio da geração, aguardando o retorno antes de continuar [?].

Para o presente trabalho, a distinção relevante é entre **quando usar** e **quando não usar** essa abordagem. O *tool use* é adequado quando se deseja que o modelo resolva autonomamente o problema durante a geração — por exemplo, chamando uma calculadora para confirmar um resultado aritmético em tempo real. Não é, porém, a abordagem adotada nesta dissertação: o sistema proposto não delega ferramentas ao LLM durante a geração, pois isso criaria uma dependência opaca entre o modelo e a ferramenta, dificultando a rastreabilidade e o controle pedagógico. Em vez disso, a verificação simbólica é realizada por um agente independente, após a geração, o que preserva a separação de responsabilidades e permite que o professor visualize tanto a questão gerada quanto o veredicto de verificação de forma transparente.

2.2.4 *Self-consistency* e amostragem múltipla

A estratégia de *self-consistency*, proposta por wang2023selfconsistency, parte de uma intuição simples: se um problema matemático admite uma resposta correta única, e se o modelo é executado múltiplas vezes com o mesmo *prompt* (com temperatura maior que

zero, para produzir variações), a resposta mais frequente entre as diferentes execuções tem maior probabilidade de ser a correta. O processo é análogo a uma votação por maioria entre raciocínios independentes.

Os resultados empíricos confirmam essa intuição. wang2023selfconsistency reportaram ganhos de 17,9% no GSM8K e de 11,0% no SVAMP ao substituir a decodificação gulosa convencional pela amostragem múltipla com votação por maioria, mantendo os demais parâmetros iguais. A técnica é agnóstica ao tipo de problema e não requer nenhum componente adicional além do próprio LLM.

A limitação principal é o **custo computacional**: k execuções do mesmo *prompt* implicam k vezes o consumo de tokens e, no caso de uso de APIs comerciais, k vezes o custo financeiro. Para $k = 10$, o custo por questão gerada pode tornar-se proibitivo em um sistema destinado a professores da rede pública com acesso limitado a recursos computacionais.

Por esse motivo, o sistema proposto nesta dissertação **não adota o *self-consistency* como mecanismo primário de verificação**. Em vez de executar o Agente Gerador múltiplas vezes e votar entre os resultados, utiliza-se uma única geração seguida de verificação simbólica determinística pelo SymPy. Essa escolha oferece uma garantia qualitativa diferente: onde o *self-consistency* reduz a probabilidade de erro por convergência estatística, a verificação simbólica detecta erros de forma determinística para as classes de questões que o verificador consegue tratar. O *custo* por questão é, assim, mantido em um patamar compatível com uso educacional real, sem renunciar à confiabilidade matemática.

Em conjunto, as quatro estratégias apresentadas formam um mapa conceitual das possibilidades disponíveis para aumentar a confiabilidade de LLMs em Matemática. O Quadro 2.1 sintetiza as escolhas feitas nesta dissertação em relação a cada uma delas.

2.2.5 Agentes e sistemas multiagentes

As estratégias de mitigação apresentadas na seção anterior — CoT, PAL, *tool use* e *self-consistency* — podem ser combinadas em sistemas mais sofisticados, nos quais diferentes componentes especializados colaboram para resolver tarefas que nenhum deles resolveria adequadamente de forma isolada. Essa ideia é formalizada pelo paradigma de **agentes e sistemas multiagentes**, que constitui a base arquitetural do sistema desenvolvido nesta dissertação.

Tabela 2.1: Estratégias de mitigação e seu papel no sistema proposto

Estratégia	Adoção	Justificativa
<i>Chain-of-thought</i>	Parcial	Usado nos <i>prompts</i> do Agente Gerador para estruturar a resolução; não suficiente sozinho.
PAL / verificação simbólica	Central	Princípio do Agente Verificador Simbólico via SymPy.
<i>Tool use</i>	Não adotado	Preservação da separação de responsabilidades e rastreabilidade pedagógica.
<i>Self-consistency</i>	Não adotado	Custo computacional incompatível com o contexto de uso; substituído por verificação determinística.

Elaborado pela autora (2024).

2.2.6 Definição de agente

Na formulação clássica de russell2021aima, um agente é qualquer entidade que **percebe** seu ambiente por meio de sensores e **age** sobre ele por meio de atuadores. O comportamento do agente é descrito por uma função que mapeia sequências de percepções em ações. Um agente é considerado **racional** quando suas ações são orientadas a maximizar uma medida de desempenho, dado o conhecimento que possui do ambiente e de suas próprias capacidades [?].

Essa abstração, originalmente desenvolvida para agentes físicos e sistemas de planejamento clássico, estende-se naturalmente aos sistemas baseados em LLMs. Um **agente baseado em LLM** percebe seu ambiente por meio de texto — um *prompt* que descreve o contexto, o histórico de ações anteriores e a tarefa corrente — e age produzindo uma resposta textual que pode ter efeitos no mundo: chamar uma ferramenta, gerar um documento, enviar uma mensagem a outro agente [?]. O LLM funciona, nesse modelo, como o módulo de decisão do agente: ele recebe a percepção e determina a ação a executar.

wang2023agentsurvey propõem uma taxonomia dos componentes de um agente baseado em LLM organizada em três dimensões. A primeira é a **memória**: a capacidade de reter e recuperar informações relevantes ao longo do tempo, que pode ser de curto prazo (o contexto da conversa atual) ou de longo prazo (um banco de dados externo consultável). A segunda é o acesso a **ferramentas**: a possibilidade de invocar módulos externos — calculadoras, interpretadores de código, mecanismos de busca, APIs — que ampliam as capacidades do LLM além do que ele consegue fazer apenas com geração

de texto. A terceira é a capacidade de **raciocínio e planejamento**: a habilidade de decompor tarefas complexas em subproblemas, executar cada etapa e revisar o plano à luz dos resultados obtidos [?].

Para os fins desta dissertação, cada agente do sistema é definido por uma combinação específica dessas três dimensões: o Agente Gerador possui acesso ao histórico de tentativas anteriores (memória) e recebe instruções detalhadas sobre o formato de saída; o Agente Verificador possui acesso ao SymPy como ferramenta; e o Agente Crítico Didático possui uma rubrica pedagógica incorporada ao seu *prompt* como conhecimento estruturado. A definição detalhada de cada agente é apresentada no Capítulo ??.

2.3 Sistemas multiagentes

Um **sistema multiagente** (SMA) é composto por dois ou mais agentes que interagem em um ambiente compartilhado, podendo cooperar, competir ou assumir papéis complementares na resolução de uma tarefa comum [?, ?]. No contexto dos LLMs, essa ideia foi retomada por wu2023autogen, que demonstraram que a colaboração entre múltiplos agentes especializados permite resolver tarefas de maior complexidade do que as que qualquer agente individual conseguiria enfrentar, ao mesmo tempo que melhora a factualidade e a robustez das saídas.

Três propriedades distinguem os SMAs de arquiteturas monolíticas e justificam sua adoção no presente trabalho.

A primeira é a **especialização**: cada agente é configurado para executar uma função bem delimitada, com um *prompt* de sistema, ferramentas e critérios de avaliação específicos para aquela função. Um agente especializado em verificação simbólica não precisa — e não deve — se preocupar com qualidade pedagógica; um agente especializado em avaliação didática não precisa reproduzir cálculos algébricos. Essa divisão de responsabilidades reduz a ambiguidade de cada tarefa e melhora a qualidade do resultado de cada componente isoladamente [?].

A segunda propriedade é a **verificação independente**: quando um agente produz um resultado que é avaliado por outro agente com critérios distintos, erros que passariam despercebidos em uma arquitetura monolítica tendem a ser detectados. Esse princípio é análogo à revisão por pares em pesquisa científica: um mesmo autor raramente detecta todas as falhas do próprio texto. No sistema desta dissertação, o Agente Gerador e o Agente Verificador operam com base em mecanismos completamente diferentes — geração probabilística versus cômputo simbólico determinístico — o que torna a verificação genuinamente independente [?].

A terceira propriedade é a **iteratividade controlada**: a presença de um orquestrador que coordena os ciclos de geração, verificação e revisão permite que o sistema refine progressivamente a qualidade da questão sem comprometer a rastreabilidade do processo. Cada iteração deixa um registro auditável, o que é especialmente importante no contexto educacional, onde o professor precisa compreender a proveniência e o grau de confiança de cada questão produzida [?].

2.3.1 Padrões contemporâneos de orquestração

A organização dos agentes em um SMA segue padrões arquiteturais que determinam como as tarefas são distribuídas, como os resultados são encaminhados entre agentes e como conflitos são resolvidos. A literatura identifica dois padrões fundamentais e um ciclo operacional central [?, ?].

O **padrão supervisor** (*supervisor pattern*) organiza os agentes em uma hierarquia com um agente coordenador que decide quais agentes subordinados acionar, em que ordem e com quais entradas. O coordenador tem visão global do processo e é responsável pela política de decisão: aprovar, rejeitar ou solicitar revisão. Esse padrão oferece controle centralizado e rastreabilidade clara, ao custo de concentrar no coordenador a lógica de orquestração [?].

O **padrão roteador** (*router pattern*) direciona cada tarefa ao agente mais adequado para tratá-la, com base em características da entrada. Em vez de um coordenador hierárquico, o roteador funciona como um despachante: classifica a demanda e a encaminha ao especialista correspondente. Esse padrão é mais adequado quando as tarefas são heterogêneas e bem delimitadas, mas oferece menor controle sobre ciclos de revisão [?].

Os **ciclos de revisão** (geração–crítica–revisão) representam o padrão operacional que combina os dois anteriores: um agente gera uma proposta, um segundo avalia criticamente essa proposta segundo critérios específicos, e o resultado da avaliação é devolvido ao gerador para refinamento, em um ciclo que se repete até que um critério de parada seja satisfeito [?, ?]. Esse padrão está diretamente relacionado à técnica de *self-refinement* e é o mecanismo central do sistema proposto nesta dissertação.

Frameworks disponíveis. Três *frameworks* de código aberto dominam atualmente a implementação de SMAs baseados em LLMs, cada um com características distintas.

O **AutoGen** [?], desenvolvido pela Microsoft Research, organiza os agentes como entidades *conversáveis* que trocam mensagens em linguagem natural e/ou código. Sua abstração central é a conversa entre agentes, o que facilita a implementação de ciclos de

revisão, mas pode tornar o fluxo de controle menos previsível em sistemas com muitos agentes.

O **LangGraph** baseia-se em uma representação explícita do fluxo de execução como um grafo dirigido, no qual os nós são agentes ou funções e as arestas representam transições condicionais. Essa abordagem oferece controle preciso sobre o fluxo e facilita a depuração, sendo especialmente adequada para sistemas com lógica de decisão complexa, como o sistema proposto nesta dissertação.

O **CrewAI** adota uma metáfora de equipe de trabalho (*crew*), na qual cada agente possui um papel, um objetivo e um conjunto de ferramentas definidos declarativamente. A abstração de alto nível facilita a prototipação rápida, embora ofereça menos controle sobre o fluxo de execução em comparação ao LangGraph.

O Quadro 2.2 sintetiza as características dos três *frameworks* em relação aos critérios relevantes para o presente trabalho.

Tabela 2.2: Comparativo de *frameworks* para sistemas multiagentes

Framework	Abstração central	Controle de fluxo	Adequação ao sistema proposto
AutoGen	Conversa entre agentes	Implícito (mensagens)	Adequado para prototipação; controle limitado
LangGraph	Grafo de execução	Explícito (grafo)	Alta aderência; controle preciso sobre ciclos
CrewAI	Equipe com papéis	Declarativo (alto nível)	Rápida configuração; menor controle granular

Elaborado pela autora (2024).

A decisão final entre LangGraph e CrewAI — os dois candidatos mais aderentes ao sistema proposto — é apresentada e justificada no Capítulo ??, após a discussão dos requisitos de implementação.

2.4 Sistemas de computação algébrica como verificadores

A discussão conduzida nas seções anteriores convergiu para um diagnóstico claro: LLMs geram texto matematicamente plausível, mas não executam cálculos com garantia de correção formal. A estratégia de verificação simbólica proposta nesta dissertação

responde a esse diagnóstico com uma classe de ferramentas que existe precisamente para isso: os **sistemas de computação algébrica** (*Computer Algebra Systems* — CAS). Esta seção apresenta o conceito, contextualiza historicamente seu desenvolvimento e detalha as capacidades do SymPy relevantes para o Ensino Médio.

2.4.1 Computação algébrica simbólica

Um sistema de computação algébrica é um programa capaz de manipular expressões matemáticas em sua forma simbólica — isto é, de operar sobre símbolos e estruturas algébricas exatas, sem recorrer a aproximações numéricas. A distinção em relação ao cálculo numérico é fundamental: enquanto uma calculadora científica ou uma biblioteca numérica como NumPy representa $\sqrt{2}$ por sua aproximação decimal (1,41421356...), um CAS mantém $\sqrt{2}$ como objeto simbólico exato, capaz de participar de operações como $\sqrt{2} \cdot \sqrt{2} = 2$ sem qualquer perda de precisão [?].

Essa distinção tem consequências práticas diretas para o contexto desta dissertação. Um gabarito que afirma que as raízes de $x^2 - 2 = 0$ são $x = \pm\sqrt{2}$ não pode ser verificado por comparação numérica sem margem de erro; é preciso verificar se $(\sqrt{2})^2 - 2 = 0$ de forma exata, o que exige manipulação simbólica. De modo análogo, confirmar que $\frac{x^2-1}{x-1} = x+1$ para $x \neq 1$ requer simplificação algébrica, não avaliação numérica em pontos isolados.

Do ponto de vista histórico, a computação algébrica simbólica tem suas raízes no projeto MACSYMA, desenvolvido no MIT na década de 1960, que foi o primeiro sistema a demonstrar a viabilidade de manipulação simbólica em larga escala por computador [?]. Nas décadas seguintes, dois sistemas comerciais tornaram-se referência: o **Maple**, desenvolvido na Universidade de Waterloo a partir de 1980 e lançado comercialmente em 1985, e o **Mathematica**, criado por Stephen Wolfram e lançado em 1988, que integrou computação simbólica, numérica e visualização em um ambiente unificado. Ambos os sistemas são amplamente utilizados em pesquisa científica e ensino superior até hoje, porém são proprietários e de licenciamento custoso, o que limita sua adoção em contextos educacionais de acesso restrito.

Esse cenário motivou o desenvolvimento do **SymPy**: uma biblioteca de computação algébrica simbólica escrita inteiramente em Python, de código aberto e licença permissiva (BSD), que não depende de nenhum sistema externo e pode ser instalada e utilizada gratuitamente em qualquer ambiente computacional [?]. Por essas características — abertura, integração nativa com Python e custo zero — o SymPy foi adotado como verificador simbólico no sistema proposto nesta dissertação.

2.4.2 SymPy: capacidades relevantes para o Ensino Médio

meurer2017sympy descrevem o SymPy como um sistema de álgebra computacional com foco em extensibilidade e facilidade de uso. Para os fins desta dissertação, interessa detalhar as capacidades do SymPy diretamente relacionadas aos conteúdos matemáticos do Ensino Médio que o sistema precisará verificar.

Resolução de equações e sistemas. O SymPy implementa a função `solve`, que recebe uma expressão ou lista de expressões e retorna o conjunto-solução exato. Para equações polinomiais de primeiro e segundo graus — as mais frequentes no Ensino Médio — o sistema retorna as raízes em forma simbólica, incluindo radicais e números complexos quando pertinente. Para sistemas lineares e não-lineares, a função `solve` com múltiplas variáveis retorna as soluções como dicionários de atribuições, permitindo comparação direta com o gabarito proposto pelo Agente Gerador [?].

Manipulação de polinômios e expressões algébricas. Operações como expansão (`expand`), fatoração (`factor`), simplificação (`simplify`) e redução ao mesmo denominador (`together`) são executadas de forma exata. Isso permite verificar, por exemplo, se a forma fatorada apresentada no gabarito é algebricamente equivalente ao polinômio original do enunciado, ou se a simplificação de uma expressão racional está correta [?].

Cálculo de derivadas, integrais e limites. Para conteúdos de introdução ao Cálculo presentes em parte dos currículos do Ensino Médio, o SymPy oferece as funções `diff` (derivada), `integrate` (integral indefinida e definida) e `limit`. Essas funções operam simbolicamente e retornam resultados exatos, o que permite verificar se a derivada ou integral apresentada no gabarito é correta sem depender de aproximações numéricas [?].

Trigonometria simbólica. O SymPy inclui funções trigonométricas simbólicas (`sin`, `cos`, `tan` e suas inversas), identidades de simplificação (`trigsimp`) e expansão de ângulos compostos (`expand_trig`). Isso permite verificar a equivalência entre expressões trigonométricas distintas que representam o mesmo valor, como $\sin^2 \theta + \cos^2 \theta = 1$ ou $\cos(2\theta) = 1 - 2\sin^2 \theta$ [?].

Equivalência simbólica e simplificação. A capacidade mais crítica para o contexto de verificação de gabaritos é a verificação de **equivalência simbólica**: dadas duas expressões A e B , determinar se $A - B = 0$ de forma exata. No SymPy, isso é realizado pela função `simplify` aplicada à diferença $A - B$: se o resultado for zero, as expressões são

equivalentes. Essa abordagem é mais robusta do que a comparação numérica em pontos isolados, pois captura identidades algébricas gerais, não apenas coincidências em valores particulares [?].

2.4.3 Limites da verificação simbólica

A despeito de suas capacidades, a verificação simbólica via SymPy apresenta limites que o sistema desta dissertação precisa reconhecer e tratar explicitamente.

O primeiro limite diz respeito a **problemas de geometria com construção**. Questões que envolvem demonstração de propriedades geométricas, construção com régua e compasso ou argumentação sobre figuras não se prestam à verificação simbólica direta: não há um conjunto-solução numérico ou algébrico a ser comparado, mas sim uma argumentação lógica cuja validade não pode ser capturada por uma expressão Python. Para esse tipo de questão, o Agente Verificador emite o veredicto *não-verificável* e devolve a responsabilidade da validação ao professor.

O segundo limite refere-se a **problemas com enunciado em linguagem natural ambíguo**. O SymPy verifica expressões matemáticas formalizadas, não enunciados em prosa. Para que a verificação ocorra, é necessário extrair do enunciado a expressão ou sistema a ser verificado, o que pressupõe que a linguagem do enunciado seja precisa e não admita múltiplas interpretações. Quando o Agente Gerador produz um enunciado ambíguo, o Agente Verificador pode formalizar uma interpretação diferente da pretendida, produzindo um veredicto incorreto. Esse risco é mitigado pelo Agente Crítico Didático, que avalia explicitamente a clareza do enunciado como um dos critérios da rubrica pedagógica.

O terceiro limite é a **incompletude dos algoritmos de simplificação**. Para certas classes de expressões — em particular, expressões envolvendo funções especiais, exponenciais e logaritmos em combinações não triviais — a função `simplify` pode não conseguir reduzir a diferença $A - B$ a zero mesmo quando as expressões são matematicamente equivalentes [?]. Nesses casos, o SymPy retorna uma expressão não nula que não é necessariamente diferente de zero.

Para tratar esse limite de forma operacional, o sistema adota uma **estratégia complementar de verificação numérica**: quando a verificação simbólica não produz veredicto conclusivo, o Agente Verificador avalia as expressões em um conjunto de pontos aleatórios gerados por amostragem e compara os resultados numéricos com tolerância definida. Essa verificação numérica em casos-teste não oferece prova de equivalência, mas constitui evidência probabilística forte: se duas expressões concordam em k pontos aleatórios independentes, a probabilidade de discordância em algum ponto

não testado decresce exponencialmente com k . O veredicto resultante é registrado como *aprovado com ressalva numérica*, sinalizando ao professor que a equivalência foi verificada empiricamente, não simbolicamente.

2.5 Síntese: a hipótese arquitetural desta dissertação

Os quatro pilares conceituais apresentados neste capítulo — os modelos de linguagem de grande escala, as estratégias de mitigação de seus erros, os sistemas multiagentes e a computação algébrica simbólica — não são contribuições isoladas. Eles se articulam em uma hipótese de trabalho que pode ser enunciada de forma direta: **a combinação de um LLM gerador, um verificador simbólico determinístico, um agente avaliador pedagógico e um orquestrador de ciclos de revisão produz questões matemáticas de maior confiabilidade do que qualquer um desses componentes poderia produzir de forma isolada.**

A lógica dessa combinação parte de uma constatação sobre limites complementares. O LLM é o único componente capaz de compreender uma especificação pedagógica em linguagem natural — tópico, habilidade BNCC, nível cognitivo, contexto temático — e produzir um enunciado plausível, uma resolução passo a passo e, no caso de questões de múltipla escolha, distratores com erros pedagogicamente informativos. Nenhum sistema determinístico realiza essa tarefa: ela exige a flexibilidade linguística que apenas o treinamento em larga escala confere. Mas essa mesma flexibilidade probabilística é a fonte do problema identificado na Seção 1.4: o LLM não sinaliza incerteza quando erra em álgebra. Ele erra com fluência, produzindo o que se denominou *erro silencioso* — uma resposta incorreta apresentada com o mesmo grau de confiança aparente de uma correta.

É exatamente nesse ponto que o SymPy intervém. O verificador simbólico não é um segundo LLM revisando o trabalho do primeiro: ele opera segundo uma lógica completamente distinta, baseada em transformações algébricas determinísticas. Quando recebe uma expressão matemática bem formada, sua resposta é exata e reproduzível, independentemente de quantas vezes o mesmo cálculo seja executado. Por isso, seus erros — quando ocorrem — são de natureza estruturalmente diferente dos erros do LLM, o que torna a probabilidade de falha simultânea dos dois componentes na mesma direção muito menor do que em qualquer arquitetura homogênea. A verificação independente por mecanismo diferente é, portanto, mais do que uma redundância: é uma garantia qualitativa que a simples repetição do LLM não ofereceria, como demonstrado na análise do *self-consistency* na Seção 2.2.4.

A correção matemática, porém, não esgota os critérios de qualidade de uma questão

destinada ao Ensino Médio. Uma questão pode estar algebricamente impecável e ser pedagogicamente inadequada: enunciado ambíguo, nível cognitivo incompatível com a habilidade BNCC declarada, distratores sem função formativa ou contexto desconectado da realidade dos estudantes. Essa dimensão é coberta pelo agente crítico didático, que avalia a questão com base em uma rubrica derivada dos referenciais da literatura de educação matemática. Esse critério externo captura problemas que nem o LLM nem o CAS conseguem detectar por si mesmos — e é precisamente por ser externo que sua avaliação tem valor: a rubrica foi construída independentemente da questão que está sendo julgada.

O orquestrador articula esses três componentes em ciclos de revisão controlados. Ele não gera, não verifica e não avalia: decide. Com base nos veredictos do verificador e do crítico, determina se a questão está aprovada, se deve retornar ao gerador com instruções específicas de melhoria ou se deve ser descartada por inviabilidade de correção dentro do número máximo de iterações. Essa política de decisão é o que transforma a arquitetura de uma pipeline linear em um sistema adaptativo, capaz de melhorar progressivamente a qualidade de cada questão antes de apresentá-la ao professor.

O resultado dessa combinação não é a eliminação do erro — nenhum sistema automatizado pode garantir isso em linguagem natural — mas a sua **visibilidade e rastreabilidade**. O professor recebe não apenas a questão produzida, mas os veredictos estruturados de cada agente: o resultado da verificação simbólica com seu grau de certeza, as notas por critério do avaliador pedagógico e o registro das iterações que levaram à versão final. Essa transparência é o que posiciona o sistema como ferramenta de apoio à autoria docente, e não como substituto do julgamento profissional do professor — retomando o argumento da Seção ??.

Para que essa arquitetura funcione, porém, há um pressuposto que ainda precisa ser explicitado: a rubrica do agente crítico precisa estar ancorada em referenciais pedagógicos reconhecidos, e os critérios de qualidade precisam ser operacionalizáveis em um *prompt* estruturado. Sem essa ancoragem, o adjetivo *didaticamente adequado* permanece vago demais para orientar uma avaliação sistemática. O Capítulo ?? cumpre exatamente essa função: apresenta os referenciais de elaboração de questões matemáticas — a perspectiva de Polya, a Teoria das Situações Didáticas, as taxonomias cognitivas de Bloom e SOLO e as habilidades de Matemática da BNCC do Ensino Médio — e os consolida em critérios de qualidade que servirão tanto à rubrica do agente crítico quanto ao instrumento de avaliação utilizado pelo painel docente no Capítulo ??.

Elaboração de questões em Matemática: referenciais didáticos

A arquitetura multiagente descrita no capítulo anterior sustenta-se sobre dois conceitos que ainda carecem de fundamentação explícita: a especificação de entrada que o professor fornece ao sistema e a rubrica utilizada pelo agente crítico didático para avaliar as questões geradas. Ambos precisam estar ancorados em referenciais teóricos da educação matemática reconhecidos pela comunidade acadêmica e coerentes com os objetivos do Ensino Médio brasileiro.

Este capítulo cumpre essa função. Apresenta, em quatro seções articuladas, os pilares pedagógicos do sistema: a tradição da resolução de problemas e a perspectiva de Polya; a Teoria das Situações Didáticas de Brousseau; as taxonomias cognitivas de Bloom e SOLO; e as habilidades de Matemática da BNCC do Ensino Médio. O capítulo encerra com a consolidação desses referenciais em critérios de qualidade operacionalizáveis, que fundamentam tanto a rubrica do agente crítico quanto o instrumento de avaliação empírica do Capítulo ??.

3.1 Resolução de problemas e elaboração de questões

A resolução de problemas ocupa posição central no ensino de Matemática há décadas. No Brasil, essa centralidade está expressa tanto nos Parâmetros Curriculares Nacionais — que já em 1998 indicavam a resolução de problemas como eixo organizador das práticas pedagógicas — quanto na Base Nacional Comum Curricular, que em 2018 reafirmou o desenvolvimento do raciocínio lógico e da capacidade de resolução de situações-problema

como competências essenciais da área de Matemática e suas Tecnologias [?]. Antes de apresentar a perspectiva de Polya, porém, é necessário estabelecer uma distinção conceitual fundamental que atravessa toda a tradição do campo: a diferença entre **exercício** e **problema**.

Um exercício é uma tarefa cuja via de resolução é imediatamente reconhecível pelo estudante: basta aplicar um algoritmo ou procedimento já aprendido, sem que haja necessidade de elaboração ou decisão estratégica. Um problema, por sua vez, é uma situação em que o resolvidor não dispõe, de imediato, de um caminho garantido para a solução; existe uma lacuna entre o que se sabe e o que se quer determinar, e superá-la exige planejamento, tentativa e raciocínio [?]. Para dante2003didatica, o exercício matemático é essencialmente uma atividade de *aplicação* de habilidade conhecida, ao passo que o problema *necessariamente envolve invenção ou criação significativa* por parte de quem o resolve.

É importante observar que essa distinção não é absoluta nem estática: uma mesma tarefa pode ser exercício para quem já domina o procedimento correspondente e problema genuíno para quem ainda não o construiu [?]. O que define o caráter de um problema não é a sua dificuldade intrínseca, mas o grau de mobilização cognitiva que ele exige de um dado estudante em um dado momento. Isso tem implicação direta para o sistema desta dissertação: ao parametrizar o nível cognitivo da questão por meio da Taxonomia de Bloom — discutida na Seção 3.2 —, o professor orienta o Agente Gerador a produzir tarefas com graus distintos de abertura e exigência estratégica, cobrindo tanto a dimensão do exercício quanto a do problema.

3.1.1 A perspectiva de Polya

Em 1945, o matemático húngaro George Pólya publicou *How to Solve It* [?], obra que se tornaria referência fundamental para a educação matemática em todo o mundo e que no Brasil circula principalmente pela tradução *A Arte de Resolver Problemas* [?]. A proposta central do livro é que o processo de resolução de problemas matemáticos pode ser descrito, ensinado e aprendido por meio de uma estrutura heurística de quatro etapas que orientam tanto o resolvidor quanto o professor que o acompanha.

A **primeira etapa é compreender o problema**. Antes de qualquer tentativa de resolução, o estudante precisa identificar com clareza o que é dado, o que é pedido e quais condições conectam os dados à incógnita. Polya sugere que o professor oriente os alunos com perguntas como: *Você compreende todos os termos do enunciado? Consegue reescrevê-lo com suas próprias palavras? Todos os dados são necessários?* [?]. Essa etapa é frequentemente subestimada em sala de aula, mas é decisiva: muitos erros em

Matemática decorrem não de dificuldades de cálculo, mas de interpretação equivocada do que foi pedido.

A **segunda etapa é elaborar um plano**. O resolvidor deve estabelecer uma conexão entre os dados disponíveis e a solução buscada, o que pode envolver reconhecer um problema análogo já resolvido, introduzir notação auxiliar, decompor o problema em partes menores ou buscar um caso particular que revele a estrutura geral [?]. É nessa etapa que reside o núcleo criativo da resolução: a capacidade de identificar qual estratégia é pertinente diante de uma situação nova.

A **terceira etapa é executar o plano**. O resolvidor coloca em prática a estratégia escolhida, verificando cada passo: *Consigo mostrar que este passo está correto? Consigo ver claramente que está correto?* [?]. A execução disciplinada do plano, com verificação passo a passo, é o que distingue uma resolução matematicamente rigorosa de uma tentativa informal.

A **quarta etapa é examinar a solução obtida**. Polya insiste que este último passo é tão formativo quanto os anteriores: ao revisitar a resolução, o estudante verifica se o resultado é plausível, se o método pode ser simplificado e se a abordagem pode ser aplicada a problemas semelhantes [?]. É nessa etapa que o aprendizado se consolida e se generaliza — e é por isso que questões bem elaboradas devem admitir uma solução verificável por via independente.

Implicações para a elaboração de questões

A heurística de Polya não foi concebida como instrumento de avaliação, mas como guia para o resolvidor e o professor. No entanto, ela implica exigências precisas para a elaboração de questões que se pretende sejam genuinamente formativas.

A primeira exigência é a **clareza do enunciado**: para que o estudante possa executar a primeira etapa — compreender o problema —, é preciso que o enunciado apresente com precisão os dados disponíveis, o que é pedido e as condições estabelecidas, sem ambiguidades que permitam múltiplas interpretações incompatíveis [?]. Um enunciado mal formulado não cria um problema mais difícil: cria um problema diferente daquele que o professor pretendia propor.

A segunda exigência é a **não-trivialidade do caminho de solução**: uma questão é pedagogicamente pobre quando o enunciado sinaliza tão explicitamente o procedimento a adotar que o estudante não precisa executar a segunda etapa de Polya — elaborar um plano. Nesses casos, a questão opera como exercício independentemente do nível de dificuldade aparente do cálculo envolvido [?].

A terceira exigência é a **determinação e verificabilidade da solução**: a quarta

etapa de Polya pressupõe que a solução pode ser examinada e confirmada por via independente. Questões com enunciados ambíguos que admitem múltiplas respostas igualmente válidas, ou questões sem solução nos parâmetros estabelecidos, violam essa exigência e não podem ser objeto de aprendizado reflexivo [?].

Essas três exigências — clareza do enunciado, não-trivialidade do caminho de solução e determinação da resposta — compõem diretamente três dos critérios da rubrica do Agente Crítico Didático e do instrumento de avaliação pelo painel docente, como detalhado na Seção 3.4. Elas também fundamentam, de forma simétrica, a necessidade do Agente Verificador Simbólico: a terceira etapa de Polya — executar e verificar cada passo — é o que o SymPy realiza de forma determinística para as classes de questões tratáveis simbolicamente, liberando o professor da tarefa de conferir manualmente cada gabarito gerado.

3.1.2 Teoria das Situações Didáticas

A Teoria das Situações Didáticas (TSD), desenvolvida pelo matemático e pesquisador francês Guy Brousseau a partir da década de 1970 e sistematizada em sua obra de referência [?], oferece um quadro teórico complementar ao de Polya para pensar a elaboração de questões matemáticas. Enquanto a heurística de Polya descreve o processo cognitivo do resolvidor individual — o que o estudante faz diante de um problema —, a TSD descreve a estrutura da situação que o professor projeta: o ambiente de aprendizagem, as relações que nele se estabelecem e as condições que tornam possível a construção do saber pelo próprio estudante [?].

O conceito fundante da teoria é o de **situação didática**, definida por Brousseau como o conjunto de relações estabelecidas, explícita ou implicitamente, entre um estudante ou grupo de estudantes, um ambiente e um sistema educativo, com a intenção de que o estudante se aproprie de um saber constituído [?]. Essa definição desloca o foco do ensino: em vez de centrar a análise no professor que transmite e no aluno que recebe, a TSD coloca no centro a *situação* — a estrutura da interação entre o estudante e o objeto de conhecimento. Dentro dessa estrutura, o conceito de *milieu* é central: Brousseau o define como tudo aquilo que age sobre o estudante ou que é acionado por ele na situação — o problema proposto, os materiais disponíveis, os dados fornecidos, as respostas já obtidas e, em situações colaborativas, os próprios colegas [?]. O *milieu* funciona como um sistema antagonista no sentido de que oferece resistência e retroalimentação às ações do estudante, permitindo que ele verifique, por seus próprios meios, se suas decisões são adequadas. É precisamente essa resistência que torna a aprendizagem possível: um *milieu* que não oferece feedback genuíno não permite ao estudante distinguir respostas corretas

de incorretas sem recorrer à autoridade do professor.

O terceiro conceito central da TSD é o **contrato didático**: o conjunto de comportamentos que o professor espera do estudante e que o estudante espera do professor, constituído em sua maior parte de cláusulas implícitas que raramente são explicitadas, mas que regulam toda a dinâmica da relação pedagógica [?]. O contrato didático é permanentemente renegociado ao longo do processo de ensino e se manifesta de forma mais visível justamente quando é rompido. Brousseau identifica nele efeitos perversos de relevância direta para a elaboração de questões, sendo o mais conhecido o **efeito Topaze**: diante de um estudante em dificuldade, o professor tende a oferecer pistas cada vez mais explícitas até que a resposta se torne evidente, transferindo para si a responsabilidade intelectual que deveria ser do estudante [?]. Transposto para a elaboração de questões, esse efeito aparece em enunciados excessivamente detalhados que conduzem o estudante à solução por um caminho previamente pavimentado, esvaziando o caráter problemático da tarefa.

Quanto à tipologia das situações, Brousseau descreve quatro tipos pelos quais uma sequência didática bem projetada deve passar [?]. Na **situação de ação**, o estudante age sobre o *milieu* mobilizando estratégias ainda implícitas: sabe *fazer*, mesmo que não saiba ainda *dizer* o que faz. Na **situação de formulação**, precisa comunicar suas estratégias a outro, o que exige que o conhecimento operacional seja explicitado em linguagem — é nessa passagem que o conhecimento começa a se tornar saber comunicável. Na **situação de validação**, deve convencer um interlocutor da validade de sua solução produzindo argumentos que se aproximam da demonstração matemática, deslocando o critério de validade da autoridade do professor para a lógica do argumento. Na **situação de institucionalização**, o professor retoma o saber construído pelos estudantes e o formaliza, conectando-o ao conhecimento matemático estabelecido; Brousseau alerta que essa fase feita cedo demais interrompe a construção do significado, e feita tarde demais consolida interpretações inexatas [?].

É necessário reconhecer que a TSD foi concebida para descrever sequências didáticas completas, e não itens avaliativos isolados. Aplicar sua tipologia diretamente à análise de uma questão individual seria um forçamento teórico. O que a teoria oferece ao presente trabalho é um vocabulário preciso para articular o que torna uma questão pedagogicamente rica ou empobrecida. À luz da TSD, uma boa questão é aquela que constrói um *milieu* informativo e resistente — fornece os dados necessários para agir, mas não antecipa a estratégia nem conduz à resposta — e que evita o efeito Topaze ao não oferecer, no próprio enunciado, as pistas que deveriam emergir do trabalho do estudante. Essas implicações se traduzem em dois critérios da rubrica do Agente Crítico Didático:

a **adequação do *milieu*** e a **ausência de condução excessiva**. Somados aos três critérios extraídos da perspectiva de Polya — clareza do enunciado, não-trivialidade do caminho de solução e determinação da resposta —, eles formam o núcleo da dimensão pedagógica da rubrica de avaliação de questões, consolidada na Seção 3.4.

3.2 Taxonomias cognitivas

Os referenciais discutidos na seção anterior — a heurística de Polya e a Teoria das Situações Didáticas — oferecem critérios qualitativos para avaliar uma questão: ela deve exigir elaboração cognitiva genuína, construir um *milieu* resistente e evitar a condução excessiva. Esses critérios, porém, não fornecem por si mesmos uma **escala** que permita graduar a complexidade das questões de forma sistemática e comunicável. Para isso, o campo da educação desenvolveu as chamadas **taxonomias cognitivas**: classificações hierárquicas dos processos mentais exigidos por diferentes tipos de tarefa, que permitem ao professor especificar, planejar e avaliar objetivos de aprendizagem com precisão.

Para o sistema desta dissertação, as taxonomias cumprem dois papéis distintos. O primeiro é compor a **especificação de entrada**: ao declarar o nível cognitivo desejado, o professor orienta o Agente Gerador sobre o grau de complexidade da questão a produzir. O segundo é compor a **rubrica de avaliação**: o Agente Crítico Didático verifica se o nível cognitivo efetivamente demandado pela questão gerada é coerente com o nível declarado na especificação. As duas subseções seguintes apresentam as duas taxonomias adotadas neste trabalho.

3.2.1 Taxonomia de Bloom revisada

A taxonomia original de objetivos educacionais foi proposta por Benjamin Bloom e colaboradores em 1956 e se tornou um dos instrumentos mais difundidos da pedagogia ocidental, influenciando décadas de planejamento curricular e elaboração de avaliações. Em 2001, Lorin Anderson — ex-aluno de Bloom — liderou uma revisão da taxonomia original que introduziu modificações estruturais importantes [?]. É essa versão revisada, conhecida como Taxonomia de Bloom Revisada (TBR), que é adotada nesta dissertação.

A principal inovação da TBR em relação à versão original é a introdução de um **framework bidimensional**: em vez de uma única hierarquia, a revisão propõe duas dimensões independentes que, combinadas, descrevem qualquer objetivo de aprendizagem [?]. A **dimensão do conhecimento** classifica *o que* o estudante precisa aprender; a **dimensão dos processos cognitivos** classifica *o que ele faz mentalmente* com esse conhecimento.

A dimensão do conhecimento compreende quatro categorias. O **conhecimento factual** refere-se a elementos isolados e discretos — terminologia, fatos específicos, convenções — que o estudante precisa conhecer para operar em uma disciplina. O **conhecimento conceitual** abrange classificações, princípios, generalizações, teorias e estruturas que organizam os fatos em sistemas coerentes. O **conhecimento procedimental** compreende algoritmos, técnicas, métodos e critérios para determinar quando usar cada procedimento. O **conhecimento metacognitivo** envolve a consciência do próprio processo de aprendizagem e das estratégias cognitivas disponíveis [?]. Para a Matemática do Ensino Médio, as categorias mais relevantes são o conhecimento conceitual — que sustenta a compreensão de funções, equações e geometria — e o conhecimento procedimental — que fundamenta a execução de algoritmos de resolução.

A dimensão dos processos cognitivos organiza-se em seis categorias hierárquicas, apresentadas em ordem crescente de complexidade. **Lembrar** corresponde à recuperação de informação armazenada na memória de longo prazo: reconhecer ou reproduzir um conceito, fórmula ou procedimento. **Entender** envolve construir significado a partir da informação — interpretar, exemplificar, classificar, resumir, inferir, comparar, explicar. **Aplicar** consiste em executar ou usar um procedimento em uma situação específica: implementar um algoritmo conhecido para resolver um problema dado. **Analisar** implica decompor o material em partes e determinar como essas partes se relacionam entre si e com a estrutura geral: diferenciar, organizar, atribuir. **Avaliar** corresponde a emitir julgamentos com base em critérios e padrões: verificar a consistência de um argumento, criticar uma solução, justificar uma escolha. **Criar** envolve reunir elementos para formar um todo novo e coerente: gerar hipóteses, planejar, produzir [?].

A hierarquia entre essas categorias não é absoluta nem linear — um estudante pode criar em um domínio em que ainda não domina todos os procedimentos —, mas indica uma progressão geral de complexidade cognitiva que orienta o planejamento de sequências de ensino e a elaboração de avaliações equilibradas.

No contexto da Matemática do Ensino Médio, a aplicação da TBR à classificação de questões revela padrões relevantes. Pesquisa conduzida por miranda2015taxonomia com professores de Matemática da Educação Básica mostrou que as questões elaboradas concentram-se maciçamente nos três primeiros níveis — lembrar, entender e aplicar — com presença muito reduzida de itens nos níveis superiores de analisar, avaliar e criar. Esse resultado converge com o diagnóstico da BNCC, que aponta a necessidade de superar o modelo de avaliação centrado na reprodução de procedimentos em direção ao desenvolvimento de competências de raciocínio, argumentação e resolução de situações-problema [?].

Para o sistema desta dissertação, a TBR opera de forma direta na especificação de entrada: o professor seleciona o nível cognitivo desejado entre as seis categorias, e esse parâmetro é incluído no *prompt* do Agente Gerador como instrução explícita. Uma questão especificada no nível **aplicar** deve exigir a execução de um procedimento conhecido em uma situação dada; uma questão no nível **analisar** deve exigir que o estudante decomponha uma estrutura matemática para identificar relações não evidentes; uma questão no nível **criar** deve exigir que o estudante construa algo novo — uma demonstração, uma generalização ou uma estratégia original de resolução. O Agente Crítico Didático, por sua vez, verifica se a questão gerada efetivamente demanda o processo cognitivo declarado, identificando casos em que uma questão rotulada como *analisar* é, na prática, uma tarefa de *aplicar* com enunciado mais longo [?].

A combinação das duas dimensões da TBR — tipo de conhecimento e processo cognitivo — é o que torna a taxonomia especialmente útil para a elaboração de questões: ela permite ao professor e ao sistema não apenas graduar a dificuldade aparente, mas especificar com precisão o que se espera que o estudante faça mentalmente ao resolver a questão, tornando o objetivo de aprendizagem subjacente explícito e avaliável.

3.2.2 Taxonomia SOLO

A Taxonomia SOLO (*Structure of Observed Learning Outcomes*) foi proposta por John Biggs e Kevin Collis em 1982 como um modelo para classificar a qualidade das respostas produzidas por estudantes, em contraposição ao foco nos objetivos de ensino que caracteriza a taxonomia de Bloom [?]. Enquanto a TBR parte da perspectiva do professor — o que se pretende que o estudante aprenda e como graduar a complexidade desse objetivo — a SOLO parte da perspectiva da resposta observada: dado o que o estudante produziu, qual é a estrutura dessa produção? Essa diferença de orientação torna as duas taxonomias complementares, e não concorrentes.

A SOLO organiza-se em cinco níveis hierárquicos que descrevem a crescente complexidade estrutural das respostas [?]. No nível **pré-estrutural**, o estudante não demonstra compreensão relevante da tarefa: a resposta é irrelevante, equivocada ou simplesmente reproduz o enunciado sem processamento. No nível **uniestrutural**, o estudante identifica um único aspecto relevante do problema, mas não o articula com outros elementos: a resposta é pontual e desconectada do todo. No nível **multiestrutural**, o estudante mobiliza vários aspectos relevantes, mas os trata de forma independente, sem estabelecer relações entre eles: há quantidade de informação, mas ainda falta integração. No nível **relacional**, os diferentes aspectos são articulados em uma estrutura coerente, de modo que o estudante demonstra compreensão integrada do

problema como um todo: as partes se relacionam e se explicam mutuamente. No nível **abstrato ampliado**, o estudante vai além dos limites do problema dado, generalizando a estrutura relacional para novos contextos, formulando princípios ou aplicando o raciocínio a domínios não previstos no enunciado [?].

A distinção fundamental que atravessa os cinco níveis é a que separa a aprendizagem **quantitativa** da **qualitativa**. A passagem do nível pré-estrutural ao multiestrutural é, em grande medida, quantitativa: o estudante vai acumulando elementos relevantes, mas sem transformar a natureza de sua compreensão. A passagem do multiestrutural ao relacional é, ao contrário, uma mudança qualitativa: não se trata de adicionar mais um elemento, mas de reorganizar os elementos já disponíveis em uma estrutura integrada [?]. Essa distinção é pedagogicamente decisiva porque revela por que estudantes com boa memória declarativa podem acumular fatos e procedimentos matemáticos e ainda assim ser incapazes de resolver situações que exigem articulação entre conceitos.

Aplicada à elaboração de questões matemáticas, a SOLO oferece um diagnóstico preciso que o modelo de Bloom não captura com a mesma clareza: ela permite distinguir questões que admitem respostas multiEstruturais corretas — em que listar os passos de um algoritmo é suficiente — de questões que exigem respostas relacionais ou de abstrato ampliado, para as quais o estudante precisa integrar conceitos, identificar estruturas e eventualmente generalizar. Uma questão de função quadrática que pede apenas o cálculo das raízes pela fórmula de Bhaskara admite uma resposta multiestrutural correta; uma questão que pede ao estudante que justifique por que o discriminante determina o número de raízes reais exige uma resposta relacional. Essa diferença não é capturada apenas pela gradação lembrar–aplicar–analisar de Bloom, pois ambas as questões podem ser classificadas no nível *aplicar* da TBR e ainda assim exigir estruturas de resposta radicalmente diferentes.

Comparando as duas taxonomias, pode-se dizer que a TBR é mais útil para a **especificação de objetivos de ensino** e para a comunicação entre professor e sistema sobre o nível cognitivo desejado — os verbos operacionais de Bloom são reconhecíveis e já fazem parte da linguagem curricular brasileira, inclusive na BNCC. A SOLO, por sua vez, é mais útil para a **avaliação da qualidade das respostas produzidas**, pois descreve o que o estudante de fato demonstra, e não apenas o que se esperava que ele fizesse [?]. Pesquisa comparativa conduzida no contexto educacional brasileiro identificou que a SOLO demonstra maior clareza descritiva dos resultados de aprendizagem, enquanto a TBR oferece maior detalhamento dos objetivos a alcançar, sendo ambas complementares no planejamento e na avaliação [?].

No sistema desta dissertação, essa complementaridade é operacionalizada de forma

direta. A TBR compõe a especificação de entrada — o professor declara o processo cognitivo desejado com base nos verbos de Bloom — e a SOLO informa o Agente Crítico Didático na avaliação da resposta gerada: uma questão é pedagogicamente rica quando sua resposta esperada exige nível relacional ou abstrato ampliado, e empobrecida quando admite resposta correta no nível multiestrutural para um objetivo declarado como *analisar* ou *criar*. Juntas, as duas taxonomias cobrem as duas perspectivas essenciais da avaliação de uma questão: o que ela pretende e o que ela efetivamente demanda.

3.2.3 Síntese operacional

A adoção conjunta da Taxonomia de Bloom Revisada e da Taxonomia SOLO nesta dissertação não é uma escolha por exaustividade teórica, mas por complementaridade funcional. Cada uma responde a uma pergunta distinta e necessária para o sistema: a TBR responde a "*qual processo cognitivo a questão deve exigir?*" — uma pergunta voltada ao design da tarefa —; a SOLO responde a "*qual é a estrutura da resposta que a questão efetivamente demanda?*" — uma pergunta voltada à qualidade do produto gerado. Usar apenas a TBR deixaria sem resposta a questão de se o nível cognitivo declarado se traduz em uma demanda estrutural coerente na questão produzida. Usar apenas a SOLO tornaria a especificação de entrada menos comunicável, pois seus níveis descrevem respostas observadas, não objetivos declaráveis a priori.

A justificativa para não adotar outras taxonomias igualmente reconhecidas — como a taxonomia de Marzano e Kendall (2007) ou o modelo de profundidade de conhecimento de Webb (1997) — reside em dois critérios práticos. O primeiro é a **compatibilidade com o vocabulário curricular brasileiro**: a TBR já é referenciada implicitamente pela BNCC, que organiza suas habilidades em torno de verbos cognitivos que correspondem aos seis processos da revisão de Anderson e Krathwohl, tornando a articulação entre especificação de entrada e habilidade BNCC imediata e intuitiva para o professor usuário do sistema [?, ?]. O segundo critério é a **aplicabilidade empírica ao contexto da Matemática do Ensino Médio**: ambas as taxonomias têm uso documentado em pesquisas brasileiras com questões do EM, o que confere respaldo empírico às escolhas de projeto [?, ?].

No sistema desta dissertação, as taxonomias operam em dois momentos distintos do fluxo de execução. Na **especificação de entrada**, o professor seleciona um dos seis processos cognitivos da TBR — lembrar, entender, aplicar, analisar, avaliar ou criar — como parâmetro obrigatório. Esse parâmetro é incorporado ao *prompt* do Agente Gerador com exemplos de verbos operacionais e descrições do tipo de demanda esperada, orientando a geração da questão. As categorias cognitivas de maior frequência esperada para o Ensino Médio — e portanto as mais testadas no sistema — são **aplicar**, **analisar**

e **avaliar**, que correspondem ao espectro de complexidade predominante nas avaliações externas brasileiras como o ENEM [?].

Na **avaliação pelo Agente Crítico Didático**, os cinco níveis da SOLO orientam um dos critérios da rubrica: o agente verifica se a estrutura da resposta esperada para a questão gerada é compatível com o processo cognitivo declarado na especificação. Uma questão especificada como *analisar* deve exigir resposta de nível relacional ou abstrato ampliado na SOLO; se o agente identificar que a resposta esperada é correta já no nível multiestrutural, isso é sinalizado como inconsistência entre o nível declarado e o nível efetivamente demandado, e o orquestrador devolve a questão ao gerador com instrução de revisão. Essa verificação cruzada — TBR na entrada, SOLO na avaliação — é o que fecha o ciclo de garantia de qualidade cognitiva do sistema, completando, junto com os critérios derivados de Polya e da TSD, o núcleo da rubrica do Agente Crítico Didático.

3.3 BNCC do Ensino Médio

Os referenciais apresentados nas seções anteriores — a heurística de Polya, a Teoria das Situações Didáticas e as taxonomias cognitivas — fornecem os critérios pedagógicos e cognitivos que fundamentam a elaboração e a avaliação de questões. Para que esses critérios se tornem operacionais no contexto do sistema desenvolvido nesta dissertação, contudo, é preciso ancorá-los ao currículo concreto do Ensino Médio brasileiro. Essa ancoragem é fornecida pela Base Nacional Comum Curricular (BNCC), homologada em 2018 [?], que define as aprendizagens essenciais a serem garantidas a todos os estudantes da Educação Básica e organiza as habilidades de Matemática em um sistema de códigos que permite ao professor especificar, de forma precisa, qual conteúdo e qual expectativa de aprendizagem estão em jogo em cada questão gerada.

3.3.1 Estrutura geral

A BNCC do Ensino Médio organiza o currículo em quatro **áreas do conhecimento**: Linguagens e suas Tecnologias, Matemática e suas Tecnologias, Ciências da Natureza e suas Tecnologias, e Ciências Humanas e Sociais Aplicadas. Cada área define **competências específicas** — que descrevem capacidades amplas a serem desenvolvidas ao longo da etapa — e as desdobra em **habilidades** concretas, identificadas por códigos alfanuméricos que permitem sua localização e referenciamento precisos [?]. A estrutura hierárquica competência-habilidade é a unidade curricular fundamental da BNCC: enquanto a competência enuncia o que o estudante deve ser capaz de fazer em

termos amplos, a habilidade especifica o que ele deve demonstrar em situações concretas de aprendizagem.

Além da formação básica comum, a BNCC do Ensino Médio prevê os **itinerários formativos**: percursos curriculares que permitem ao estudante aprofundar-se em uma ou mais áreas do conhecimento de acordo com seus interesses e projeto de vida [?]. Os cinco itinerários possíveis correspondem às quatro áreas do conhecimento acrescidas de Formação Técnica e Profissional. No âmbito desta dissertação, o recorte recai sobre a formação básica comum da área de Matemática e suas Tecnologias, que é obrigatória para todos os estudantes do Ensino Médio e constitui o escopo das habilidades utilizadas como parâmetro de entrada no sistema. Os itinerários formativos, por sua natureza de aprofundamento opcional e de implementação heterogênea entre redes de ensino, estão fora do escopo do sistema proposto.

A estrutura geral da BNCC tem uma consequência direta para o sistema desta dissertação: o código de habilidade — da forma **EM13MATxxx**, em que os dois primeiros algarismos após **MAT** identificam a competência específica e os últimos identificam a habilidade dentro dela — funciona como identificador único e não ambíguo da expectativa de aprendizagem associada a cada questão gerada. Isso permite rastrear, para qualquer questão produzida pelo sistema, qual conteúdo ela aborda, a qual competência ela se vincula e como ela se posiciona no currículo do Ensino Médio.

3.3.2 Habilidades de Matemática

A área de Matemática e suas Tecnologias na BNCC do Ensino Médio organiza suas habilidades em torno de quatro **unidades temáticas**: Números e Álgebra, Geometria e Medidas, e Probabilidade e Estatística [?]. Cada unidade temática agrupa habilidades relacionadas a um campo conceitual da Matemática, permitindo ao professor identificar rapidamente o conjunto de conteúdos ao qual uma determinada habilidade pertence.

A unidade **Números e Álgebra** abrange habilidades que envolvem a interpretação e o uso de funções polinomiais, exponenciais, logarítmicas e trigonométricas; progressões aritméticas e geométricas; e a análise crítica de situações que envolvem variação de grandezas. É a unidade com maior densidade de habilidades no Ensino Médio e aquela que oferece maior abrangência para a geração de questões com diferentes níveis cognitivos, desde a aplicação direta de algoritmos até a interpretação e modelagem de situações reais [?].

A unidade **Geometria e Medidas** contempla habilidades relacionadas à geometria plana e espacial, grandezas proporcionais, semelhança de figuras, trigonometria no triângulo retângulo e no círculo, e transformações geométricas. Do ponto de vista da

verificação simbólica, é a unidade que apresenta maior proporção de habilidades de difícil formalização computacional, particularmente aquelas que envolvem argumentação geométrica e construção com régua e compasso — o que corresponde diretamente ao limite da verificação simbólica discutido na Seção 2.4.3.

A unidade **Probabilidade e Estatística** reúne habilidades que envolvem análise combinatória, cálculo de probabilidade de eventos simples e compostos, distribuições de frequência, medidas de tendência central e de dispersão, e interpretação de dados em diferentes representações. É a unidade com maior potencial para questões contextualizadas em situações do cotidiano, alinhadas à ênfase da BNCC na aplicação da Matemática à leitura crítica da realidade social [?].

Para o recorte desta dissertação, o sistema foi desenvolvido e testado com foco prioritário nas habilidades das unidades de Números e Álgebra e de Probabilidade e Estatística, por duas razões complementares. A primeira é a maior cobertura da verificação simbólica via SymPy para essas unidades — equações, funções, sequências e expressões algébricas admitem verificação determinística na grande maioria dos casos. A segunda é a representatividade dessas unidades nas avaliações externas brasileiras, em particular no ENEM, que constitui o principal instrumento de avaliação do Ensino Médio no país [?]. As habilidades de Geometria e Medidas que admitem verificação simbólica — como as relacionadas a equações de retas, circunferências e distâncias na geometria analítica — também foram incluídas no escopo, com os limites e ressalvas discutidos na Seção 2.4.3.

3.3.3 Articulação entre BNCC e taxonomias cognitivas

A BNCC e a Taxonomia de Bloom Revisada operam em níveis de abstração distintos, mas complementares. As habilidades da BNCC especificam *o que* o estudante deve aprender — o conteúdo e o contexto da aprendizagem esperada. A TBR especifica *como* ele deve processar cognitivamente esse conteúdo — o verbo que descreve a operação mental exigida. Essa distinção é fundamental para o sistema desta dissertação: o código de habilidade BNCC e o nível cognitivo de Bloom são dois parâmetros independentes na especificação de entrada, e sua combinação é o que permite ao professor solicitar, com precisão, uma questão sobre um conteúdo específico em um nível de complexidade determinado.

A articulação entre os dois sistemas, porém, não é arbitrária: os verbos usados nas próprias habilidades da BNCC já carregam implicitamente uma expectativa de nível cognitivo que pode ser lida à luz da TBR. Uma habilidade que usa o verbo *interpretar* aponta para o nível *entender*; uma que usa *resolver e elaborar problemas* combina *aplicar*

e eventualmente *analisar*; uma que usa *investigar* ou *analisar criticamente* aponta para os níveis superiores da taxonomia [?, ?]. Essa correspondência não é biunívoca — a mesma habilidade pode ser avaliada em diferentes níveis cognitivos dependendo do tipo de questão proposta — mas fornece uma ancoragem que orienta tanto o professor na especificação de entrada quanto o Agente Crítico Didático na avaliação da coerência entre habilidade declarada e nível cognitivo da questão gerada.

No sistema desta dissertação, essa articulação é operacionalizada da seguinte forma. O professor fornece, na especificação de entrada, o código de habilidade BNCC e o nível cognitivo desejado segundo a TBR. O Agente Gerador recebe ambos os parâmetros e os incorpora ao *prompt* com instruções explícitas: o código de habilidade determina o conteúdo e o contexto da questão; o nível cognitivo determina o tipo de demanda que o enunciado deve criar. O Agente Crítico Didático, por sua vez, verifica a coerência entre os dois parâmetros declarados e a questão efetivamente gerada, sinalizando inconsistências como uma questão de nível *analisar* que na prática demanda apenas *aplicar*, ou uma questão cujo conteúdo não corresponde à habilidade BNCC declarada.

Essa dupla ancoragem — curricular pela BNCC e cognitiva pela TBR — é o que confere ao sistema sua especificidade pedagógica. Ela permite que o professor não apenas obtenha questões matematicamente corretas, mas questões alinhadas ao currículo obrigatório do Ensino Médio e calibradas ao nível de complexidade que ele efetivamente pretende avaliar. A consolidação dessas articulações em critérios operacionalizáveis é o objeto da seção seguinte.

3.4 Critérios de qualidade para questões matemáticas

As seções anteriores apresentaram quatro referenciais teóricos distintos — a perspectiva de Polya, a Teoria das Situações Didáticas, as taxonomias cognitivas de Bloom e SOLO, e as habilidades da BNCC — cada um contribuindo com um ângulo específico para a compreensão do que torna uma questão matematicamente e pedagogicamente adequada. Esta seção consolida essas contribuições em um conjunto coeso de seis critérios de qualidade que operam simultaneamente como rubrica do Agente Crítico Didático e como instrumento de avaliação pelo painel docente descrito no Capítulo ???. Os critérios não são independentes entre si: eles se articulam em uma estrutura que vai da dimensão mais técnica — a correção matemática — à dimensão mais pedagógica — a relevância formativa da questão.

1. Correção matemática. Uma questão de Matemática só cumpre sua função formativa se os dados apresentados são consistentes, o gabarito está correto e a resolução

proposta não contém erros. Esse critério é o mais fundamental e o único que admite avaliação binária: a questão está matematicamente correta ou não está. No sistema desta dissertação, é o único critério avaliado de forma determinística, pelo Agente Verificador Simbólico via SymPy, antes mesmo de a questão chegar ao Agente Crítico Didático. Uma questão reprovada por correção matemática é devolvida ao Agente Gerador com o veredicto e a justificativa do verificador, sem passar pelos demais critérios. Questões com enunciados que admitem múltiplas interpretações matemáticas incompatíveis, ou cujo gabarito depende de dados omitidos, também são reprovadas neste critério, pois violam a exigência de Polya de que a solução seja determinada e verificável [?].

2. Clareza do enunciado e ausência de ambiguidade. Um enunciado claro é aquele que permite ao estudante identificar sem dúvida o que é dado, o que é pedido e quais são as condições — a primeira etapa da heurística de Polya [?]. A ambiguidade pode ser lexical (um termo com dois significados matemáticos distintos), estrutural (uma frase que admite duas leituras diferentes) ou de contexto (uma situação descrita de forma incompleta que não fornece ao estudante os dados suficientes para agir). À luz da TSD, a ambiguidade empobrece o *milieu*: quando o estudante não consegue determinar sobre qual problema está sendo solicitado a raciocinar, a resistência produtiva do ambiente se dissolve em confusão [?]. O Agente Crítico Didático avalia este critério examinando se o enunciado gerado é preciso na terminologia, completo nos dados fornecidos e inequívoco na formulação do que é pedido.

3. Adequação ao nível cognitivo e ao público. Uma questão é adequada ao nível quando o processo cognitivo que ela efetivamente demanda é coerente com o nível declarado na especificação de entrada e compatível com o repertório esperado de um estudante do Ensino Médio. Esse critério opera em duas dimensões simultâneas. Na dimensão cognitiva, o Agente Crítico verifica, com base na Taxonomia SOLO, se a estrutura de resposta exigida pela questão é compatível com o nível Bloom declarado: uma questão especificada como *analisar* deve demandar resposta relacional ou de abstrato ampliado, não multiestructural [?, ?]. Na dimensão curricular, o agente verifica se os conceitos e procedimentos envolvidos na resolução estão previstos para o Ensino Médio, evitando questões que pressuponham conhecimentos de nível universitário ou que sejam trivialmente elementares para a etapa.

4. Alinhamento à habilidade BNCC declarada. Cada questão gerada pelo sistema está associada a um código de habilidade BNCC fornecido pelo professor na especificação de entrada. O alinhamento é avaliado em dois planos. No plano do conteúdo, a questão deve abordar o objeto de conhecimento correspondente à habilidade declarada: uma questão associada à habilidade EM13MAT302, por exemplo, deve envolver

efetivamente geometria analítica, e não outro conteúdo. No plano do verbo, a demanda cognitiva da questão deve ser compatível com a expectativa expressa na habilidade BNCC — que, como discutido na Seção 3.3.3, já carrega implicitamente uma orientação sobre o nível cognitivo esperado [?]. Inconsistências entre conteúdo, habilidade e nível cognitivo são sinalizadas pelo agente como indicação de revisão.

5. Plausibilidade dos distratores (em questões de múltipla escolha). Quando o formato solicitado pelo professor é o de múltipla escolha, a qualidade da questão depende não apenas do enunciado e do gabarito, mas também dos distratores — as alternativas incorretas. Um distrator bem elaborado é plausível: corresponde a um erro sistemático real que estudantes cometem ao processar aquele conteúdo, como confundir o discriminante com a raiz, inverter numerador e denominador em uma fração algébrica ou esquecer de considerar todos os casos em um problema de probabilidade [?]. Um distrator implausível — aquele que nenhum estudante razoável escolheria — não cumpre função diagnóstica e reduz artificialmente a dificuldade da questão. O Agente Crítico avalia se cada distrator gerado corresponde a um erro matematicamente coerente com o raciocínio do estudante, e não a um valor aleatório ou absurdo.

6. Originalidade e relevância pedagógica. Uma questão é original quando não reproduz mecanicamente enunciados de livros didáticos ou de provas anteriores, e relevante quando o contexto escolhido para a situação-problema é significativo para o estudante — seja por sua conexão com situações reais, seja por iluminar uma propriedade matemática de forma genuinamente formativa. À luz da TSD, a relevância pedagógica está diretamente ligada à qualidade do *milieu*: um contexto rico e autêntico cria condições para que o estudante se engaje com o problema como uma situação de ação genuína, e não como um exercício de reconhecimento de padrão [?]. Este é o critério de avaliação mais subjetivo dos seis, e por isso é avaliado pelo Agente Crítico com uma rubrica descritiva em escala de três pontos — inadequado, aceitável, rico — em vez de um veredicto binário.

O Quadro 3.1 sintetiza os seis critérios, seus referenciais teóricos de origem, o mecanismo de avaliação no sistema e o tipo de resposta produzida pelo agente ou verificador correspondente.

Tabela 3.1: Critérios de qualidade, referenciais e mecanismos de avaliação

Critério	Referencial	Agente avaliador	Tipo de resposta
Correção matemática	Polya; SymPy	Verificador Simbólico	Binário (aprovado / rejeitado)
Clareza do enunciado	Polya; TSD	Crítico Didático	Escala de comentário +
Adequação ao nível	Bloom; SOLO	Crítico Didático	Escala de comentário +
Alinhamento BNCC	BNCC; Bloom	Crítico Didático	Binário justificativa +
Plausibilidade dos distratores	Dante; TSD	Crítico Didático	Escala por distrator
Originalidade e relevância	TSD; Polya	Crítico Didático	Escala descritiva (3 pontos)

Elaborado pela autora (2024).

Os seis critérios formam a rubrica operacional do sistema e serão retomados no Capítulo ?? como instrumento de avaliação pelo painel docente: os professores participantes da pesquisa empírica receberão uma versão desta rubrica adaptada para avaliação humana, permitindo a comparação direta entre o veredicto do Agente Crítico Didático e o julgamento dos especialistas. Essa triangulação — entre verificação simbólica determinística, avaliação automatizada por rubrica e avaliação humana qualificada — é o mecanismo de validação da hipótese arquitetural descrita na Seção 2.5.